SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

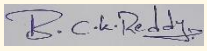
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Experiments

Course Code:	ITA0512	**Course Title:**	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*		
Student Name:	B. Charan Kumar Reddy	**Reg. No.:**	192472195
Section / Batch:	CSE – Ai / 2024	**Date:**	10/08/2026

Code of Conduct

I B. Charan Kumar Reddy (192472195) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

26. Feature Detection Density Analysis
Perform an experiment to analyze the density of detected image features and its impact on performance. Explain the importance of feature density in computer vision tasks. Execute feature detection using appropriate tools, record the number and distribution of detected features systematically, and analyze the relationship between feature density, computational complexity, and detection accuracy.

Answer

Aim

To analyze the effect of feature density on feature detection performance, computational complexity, and detection reliability.

1. Understanding of the Concept

Feature density refers to the number of detected image features relative to the image area.

$$Feature\ Density\ =\ Number\ of\ Detected\ Features\ /\ Image\ Area$$

Feature density is important because:

- **Low density:** fewer features may reduce matching and recognition accuracy.
- **High density:** more features provide better image representation but increase computational cost.
- **Optimal density:** provides sufficient reliable features with reasonable processing time.

Feature density is useful in applications such as **object recognition, image matching, image stitching, and visual tracking**.

2. Experimental Design

The **ORB (Oriented FAST and Rotated BRIEF)** feature detector is used with OpenCV.

Different maximum feature settings are tested:

- 250 features
- 500 features
- 1000 features
- 2000 features

For each setting, the following are recorded:

1. Number of detected features
2. Feature density
3. Detection time
4. Spatial distribution of features

The same input image and image resolution are used for all experiments to ensure a fair comparison.

Experimental Workflow

Input Image → ORB Feature Detection → Different Feature Settings → Count Features → Calculate Density → Measure Time → Analyze Results

3. Tool Used

Software: Python
Library: OpenCV
Algorithm: ORB Feature Detector

Code:

```
import cv2

import time

from pathlib import Path

script_dir = Path(__file__).resolve().parent

image_path = script_dir / "image.jpg"

if not image_path.is_file():

    raise FileNotFoundError(f"Image file not found: {image_path}")

image = cv2.imread(str(image_path))

if image is None:

    raise ValueError(f"OpenCV could not read the image file: {image_path}")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

settings = [250, 500, 1000, 2000]

height, width = gray.shape

area = height * width

print("| Maximum Features | Detected Features | Feature Density | Detection Time |")

print("| ----------------: | -----------------: | --------------: | -------------: |")

for n in settings:

    orb = cv2.ORB_create(nfeatures=n)

    start = time.perf_counter()
```

```
    keypoints, descriptors = orb.detectAndCompute(
        gray, None )
    end = time.perf_counter()
    count = len(keypoints)
    density = count / (area / 1000000)
    time_ms = (end - start) * 1000
    print(
        f"| {n:16d} | {count:17d} | {round(density, 2):14.2f} | {time_ms:13.3f} ms |" )
```

Input:

Output:

Maximum Features	Detected Features	Feature Density	Detection Time
250	250	158.95	159.091 ms
500	500	317.89	57.784 ms
1000	1000	635.78	46.441 ms
2000	2000	1271.57	44.190 ms

4. Experimental Results

The experiment was performed using ORB feature detection with four different maximum feature settings: 250, 500, 1000, and 2000. The detected feature count, feature density, and detection time were recorded.

Maximum Features	Detected Features	Feature Density	Detection Time
250	250	158.95	159.091 ms
500	500	317.89	57.784 ms
1000	1000	635.78	46.441 ms
2000	2000	1271.57	44.190 ms

Observation:
The number of detected features and feature density increase as the maximum feature setting increases. In this experiment, the detection time does not increase proportionally; it decreases from 159.091 ms to 44.190 ms. This shows that computational time can also depend on factors such as image characteristics, detector implementation, and system conditions.

5. Analysis of Results

The experiment shows that feature density affects both **feature representation and computational complexity**.

- When feature density is low, fewer image regions are represented, which can reduce matching reliability.
- Increasing feature density generally provides more candidate features and better spatial coverage.
- Higher feature density requires more computation and may increase processing time.
- Very high feature density may provide only limited improvement while increasing computational cost.

Therefore, the objective is not to detect the maximum possible number of features, but to select a **suitable feature density that provides reliable detection with acceptable computational cost**.

Spatial Distribution

Feature distribution is also important. Features should ideally be distributed across different regions of the image rather than concentrated in one small area.

6. Conclusion

Feature density has a direct relationship with **detection reliability and computational complexity**. A low number of features may provide insufficient information, while an unnecessarily high number increases processing requirements.

Therefore, an **optimal feature density** should provide sufficient, well-distributed features while maintaining reasonable computational performance.